Reflected XSS with AngularJS Sandbox Escape and CSP

1. Overview

Field	Details
Vulnerability	Reflected Cross-Site Scripting
Technology	AngularJS
Attack Technique	Sandbox Escape and CSP Bypass
Security Control	Content Security Policy
Injection Context	AngularJS Expression
Severity	Context-dependent
Lab Platform	PortSwigger Web Security Academy

2. Description

This vulnerability occurs when user-controlled input is reflected into an AngularJS expression context and evaluated by the browser.

The application attempts to restrict script execution through AngularJS sandbox restrictions and a Content Security Policy (CSP).

However, legacy AngularJS expression evaluation and weaknesses in the application's security configuration may create opportunities for attackers to bypass these protections.

This lab demonstrates how multiple client-side security mechanisms can be undermined when untrusted expressions are evaluated.

3. Root Cause

The underlying weaknesses include:

- Unsafe evaluation of user-controlled AngularJS expressions.

- Reliance on AngularJS sandbox restrictions.
- Inadequate CSP configuration or assumptions about CSP protection.
- Insufficient separation of application data and executable code.

CSP is a defense-in-depth mechanism, not a replacement for XSS prevention.

4. Proof of Concept (PoC)

Authorized lab testing methodology:

1. Open the assigned lab.
2. Identify the reflected input parameter.
3. Inspect the AngularJS expression context.
4. Examine the response headers and identify the active CSP.
5. Determine which script execution restrictions are enforced.
6. Investigate whether the AngularJS expression context permits unsafe evaluation.
7. Verify the result using a harmless expression and browser developer tools.
8. Document the relevant CSP restrictions and observed behavior.

Example expression test:

```
None
{{1+1}}
```

This verifies expression evaluation only; it is not a sandbox escape or CSP bypass payload.

5. Security Impact

Successful exploitation may result in:

- JavaScript execution within the application's origin.
- Bypass of intended client-side security restrictions.
- Manipulation of application content.
- Unauthorized actions through authenticated browser sessions.
- Exposure of information accessible to client-side scripts.

6. Recommended Remediation

- Never compile or evaluate untrusted AngularJS expressions.
- Remove unsafe dynamic template compilation.
- Upgrade legacy AngularJS applications.
- Use strict CSP policies with nonces or hashes where appropriate.

- Avoid unsafe-inline and unsafe-eval wherever possible.
- Restrict permitted script sources.
- Apply contextual output encoding.
- Use CSP reporting to identify policy violations.

7. Verification

1. Retest the original input.
2. Confirm that untrusted expressions are not evaluated.
3. Verify CSP enforcement through browser developer tools.
4. Check for unexpected script execution.
5. Confirm that application functionality remains operational.

8. References

- PortSwigger: XSS Contexts
- AngularJS Security Guide
- MDN: Content Security Policy
- OWASP XSS Prevention Cheat Sheet

9. Disclaimer

For educational purposes and authorized security testing only.